

Java – Beautifully Broken

Objects that forget who they are, and a free licence that runs out on 20 October

Issue #1

Mon, 05 Oct 2026

~33 min read

Level 3/5

Today's two stories

1. **Project Valhalla's value objects are officially headed for JDK 28 as a preview.** This is the biggest change to what a Java object is since 1995. `==` starts comparing contents, `synchronized` on an `Integer` stops working, and arrays of small objects can finally be laid out flat in memory.
2. **Oracle JDK 21 stops being free for production on 20 October 2026.** The October Critical Patch Update ships under a different licence. If your servers run Oracle's own JDK 21 binaries, you have about two weeks to choose: upgrade, switch distributions, or pay.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 Value objects are coming: Java's biggest object-model change in 30 years

JDK 28 · OPENJDK · PROJECT VALHALLA · TARGETED FOR JDK 28 (PREVIEW) · ~14 MIN READ

Here's a line of Java you have probably written, or at least reviewed: `if (a == b)` where `a` and `b` are both `Integer`. It works in your unit test with small numbers, then fails in production with 1,000. Every Java developer learns this lesson once. In JDK 28 the lesson changes: with preview features on, two `Integer`s holding 1,000 *are* `==`. The reason isn't a tweak to `Integer`. It's a new kind of object that has no identity at all, and it took the OpenJDK team twelve years to get here.

WHAT HAPPENED

JEP 401, "Value Objects (Preview)", is now targeted to JDK 28, the release due in March 2027.¹ It's the first piece of **Project Valhalla** *≈ the long-running OpenJDK project to give Java objects the memory efficiency of primitives like `int`* to reach a mainline JDK. The code itself is enormous. When Oracle engineer Lois Foltan announced the plan to merge it in June, the pull request added more than 197,000 lines across 1,816 files, and she called it an "extremely large change".² The work was split into three separate reviews (language, JVM, libraries) under one umbrella pull request, which also carries a companion JEP.³

That companion is JEP 539, "Strict Field Initialization in the JVM" (Preview). Both JEPs moved to *Targeted* for JDK 28 in late July,⁴ and InfoQ reported on 10 August that JEP 401 had been integrated into the JDK 28 codebase.⁵ Inside.java, the Java team's official news site, published the targeting notice on 20 September.¹

Like every preview feature, it's **off by default**. You need `--enable-preview` when you compile *and* when you run.⁵ Brian Goetz, Oracle's Java language architect, has already said not to expect a quick graduation: the

feature "will still likely be in preview in the next LTS release" (JDK 29, September 2027).²

THE BACKGROUND YOU NEED

Every object has a face

Since Java 1.0, every object you create with `new` has **identity**. **Identity** *≈ the property that makes an object distinguishable from every other object, even one with exactly the same field values. It is what `==` checks for objects today* is why `new String("hi") == new String("hi")` is `false`: two allocations, two distinct objects, two faces.

Identity is what lets an object have mutable state that other code can watch change. It's what makes `synchronized (obj)` possible, because the JVM needs a specific object to attach a lock to. And it's what `System.identityHashCode` hands you.

It also has a cost. As the original design notes put it, "support for identity can be expensive. It generally requires that an object's data be located at a particular memory location, packaged with metadata."⁶ That metadata is the **object header** *≈ the bookkeeping bytes the JVM stores at the start of every object, holding lock state, GC information and a pointer to the object's class*, and the "particular memory location" means every object lives at its own address on the **heap** *≈ the region of memory where the JVM stores objects created with `new`, managed by the garbage collector*.

The cost you pay without noticing

Picture a `Point[]` with a million points. Today that array doesn't contain points. It contains a million **references** *≈ pointers to where an object lives on the heap. A variable of a class type holds a reference, not the object itself*, each pointing to a separate little object somewhere on the heap, each with its own header. Walking the array means jumping all over memory. Modern CPUs hate that. They're fast when data sits next to each other in a **cache line** *≈ the small chunk of memory, typically 64 bytes, that a CPU loads at once. Reading one byte pulls in its neighbours for free*, and slow when every element costs a trip to main memory.

The alternative today is ugly: drop the class and use two `int[]` arrays, one for x and one for y. It's fast, but, in the designers' words, "if a geographic location is encoded as two floats, there's no way to restrict valid ranges or keep matching pairs together."⁶ You lose validation, methods, type safety: everything that made it a class.

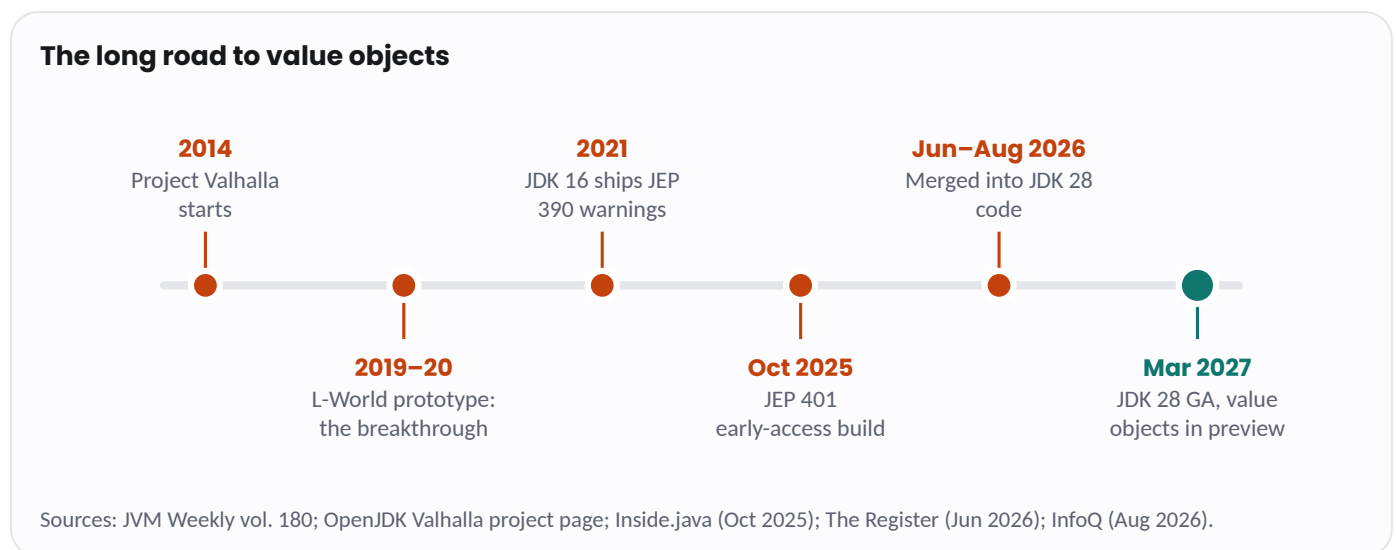
Plenty of classes never use identity anyway. `Integer`, `LocalDate`, `Optional`, a `Money` type in your billing code: nobody locks on them, nobody mutates them, and nobody *should* care whether two equal ones are the same allocation. The JVM can't know that, so it pays for identity regardless. It does have a trick, **escape analysis** *≈ a JIT optimisation that spots objects which never leave a method and avoids allocating them on the heap*, but the designers call it "somewhat unpredictable", and it does nothing for objects stored in fields and arrays,⁶ which is exactly where the million points live.

Twelve years, five prototypes, four names

Valhalla started in 2014. Its slogan from day one has been "**codes like a class, works like an int**".⁷ The road was long because the problem is hard: James Gosling reportedly described it as "six PhDs tied into a single

knot".⁷ The team built five prototypes. The breakthrough, around 2019–2020, was the "L-World" model, which kept value types inside the ordinary object world instead of inventing a parallel type system.⁷ The concept went through four names: "value types", then "inline classes", then "primitive classes" (with confusing `Point.val` / `Point.ref` syntax that was dropped as "mentally heavy"), and finally **value classes** whose instances are **value objects**.⁷ Even the JEP's own title has changed: it shipped as an early-access build in October 2025 as "Value Classes and Objects (Preview)",⁸ and is targeted to JDK 28 as "Value Objects (Preview)".¹

Valhalla has been quietly preparing your code for years. JEP 390, delivered in JDK 16, added warnings for **value-based classes** *≈ JDK classes such as `Integer`, `LocalDate` and `Optional` whose documentation already said: don't rely on the identity of these objects*,⁹ precisely "in anticipation of this feature".⁶ You can see those warnings on a plain JDK 21 today (we'll do that in a minute).



HOW IT WORKS

One new keyword

A **value class** *≈ a class declared with the value modifier. Its instances have no identity, so they are distinguished only by their field values* is declared with a new **contextual keyword** *≈ a word that is a keyword only in one specific position, so existing code using it as a variable name still compiles*, `value`. The rules for a value class all follow from "it has no identity":⁵

- All instance fields are implicitly `final`, and every field must be assigned before the constructor finishes.
- No `synchronized` instance methods. With no identity, there's nothing to lock.
- A concrete value class is implicitly `final`, so you can't subclass it.⁷

Everything else still works. A value class can have methods, implement interfaces, and validate its arguments in the constructor. It's still a class, and it's still a **reference type** *≈ a type whose variables hold references (or null), unlike primitive types such as `int`*: `Point p = null;` is perfectly legal in JDK 28.⁷

Here's how that looks next to the class you'd write today, following the JEP's own example:⁵

An immutable point: today vs JDK 28 preview

Today: an identity class

```
final class Point {  
    private final int x;  
    private final int y;  
    Point(int x, int y) {  
        this.x = x; this.y = y;  
    }  
    int x() { return x; }  
    int y() { return y; }  
}  
  
new Point(3,4) == new Point(3,4)  
  
// false: two objects
```

JDK 28 + --enable-preview

```
value class Point {  
    private int x; // implicitly final  
    private int y;  
    Point(int x, int y) {  
        this.x = x; this.y = y;  
    }  
    int x() { return x; }  
    int y() { return y; }  
}  
  
new Point(3,4) == new Point(3,4)  
  
// true: same class, same fields
```

Right pane follows the example in JEP 401 as reported by InfoQ (10 Aug 2026). It needs a JDK 28 build with the enable-preview flag on for both javac and java; we couldn't compile it here because no JDK 28 build was reachable from our sandbox. Records work too: value record Color(byte red, byte green, byte blue) { }

What == means now

For a value object, == asks a new question. Not "is this the same allocation?" but "could I swap one for the other without anyone noticing?". InfoQ's summary of the JEP: == "succeeds when both operands are instances of the same class with the same field values; reference-typed fields are compared recursively using ==".⁵ Primitive fields are compared by their bit patterns.⁶ This idea is called substitutability ≈ two values are substitutable when no program can tell them apart, so either can stand in for the other.⁷

```
// JDK 28 + --enable-preview (from the JEP, via InfoQ)  
Point p1 = new Point(3, 4);  
Point p2 = new Point(3, 4);  
assert p1 == p2;    // true: same class, same field values  
  
Object o1 = p1, o2 = p2;  
assert o1 == o2;    // true: still indistinguishable as Object  
  
String s1 = "hamburger";  
String s2 = new String(s1);  
assert s1 != s2;    // true: String remains an identity class
```

Notice the last case: String is not becoming a value class, so string comparison with == stays as broken as you remember. And the JEP is explicit that this "does not redefine == as a replacement for equals"; the usual advice to compare objects using equals still applies.⁵

There's a new way to ask the question directly, too. The early-access builds added

```
Objects.hasIdentity(obj) : Objects.hasIdentity("abc") is true , while
```

```
Objects.hasIdentity(Integer.valueOf(123)) is false once preview is enabled.8
```

Your JDK classes change too

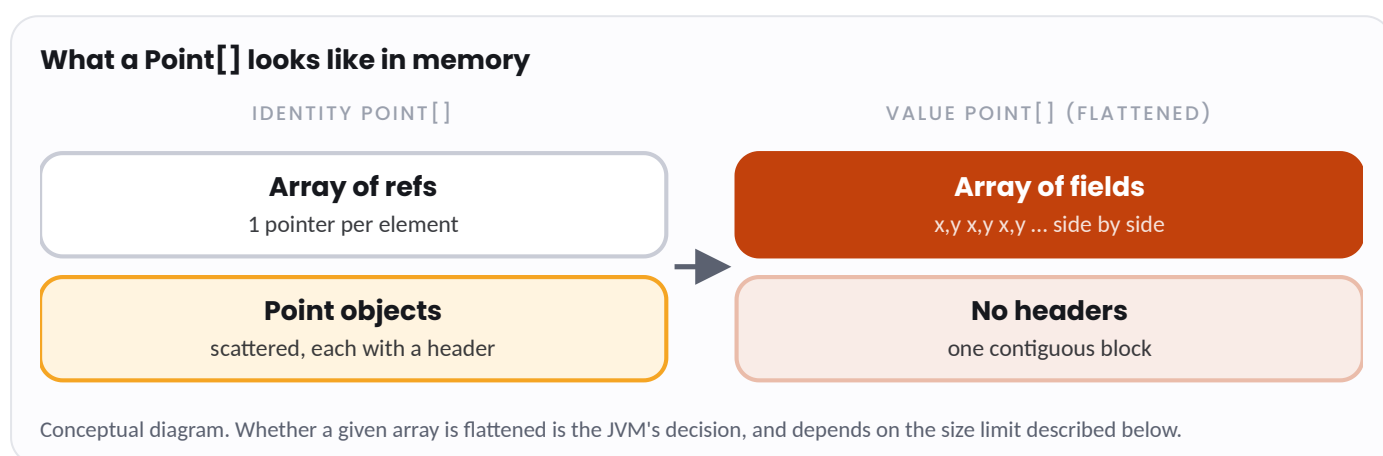
This is the part that reaches into code you didn't write. With preview features enabled, a set of existing value-based JDK classes become value classes, **including the primitive wrappers (Integer , Long , Double and friends) and LocalDate**.⁵ Run without `--enable-preview` and they stay ordinary identity classes, exactly as in JDK 27.⁵

That's why the `Integer == Integer` trap from the opener disappears in preview mode. It's also why Goetz warned that "code that synchronizes on `Integer` objects now fails with an exception."² An earlier design draft spelled out the mechanism: the compiler rejects `synchronized` on a value class type, and at run time "attempting to synchronize on a value object... results in an exception", an `IllegalMonitorStateException`.⁶

Where the speed comes from

Removing identity is a promise to the JVM: "you may copy me, split me up or store me inline, because nobody can tell the difference." The JVM then has two big optimisations available.⁵

1. **Scalarization** *≈ the JIT compiler breaking a value object into its separate fields and keeping them in CPU registers, so the object is never allocated on the heap* . Inside compiled code, a `Point` becomes two `int` s in registers. No allocation, no garbage. This is what escape analysis tries to do today, but now it doesn't depend on proving the object never escapes.
2. **Heap flattening** *≈ storing a value object's fields directly inside the field or array element that holds it, instead of storing a pointer to a separate object* . A `Point[]` can store the points' fields side by side, with no per-element header and no pointer chasing.



The Valhalla team showed what this buys in the October 2025 early-access build. They filled an array with 50 million `LocalDate` s and summed the years. Without preview, five runs took 82.7 to 71.9 ms. With preview on, the same code took 42.0 down to 25.0 ms, which they described as a nearly 3x speedup from heap flattening.⁸

Summing years over 50 million LocalDates (5th run, ms, lower is better)

Identity (no preview)

71.9 ms

Value (preview on)

25.0 ms

Numbers from the Valhalla team's own demo on the JDK 26-based JEP 401 early-access build (Inside.java, 27 Oct 2025). One micro-benchmark by the feature's authors: treat it as a claim, not a prediction for your workload.

The catch: nulls and torn reads

Flattening isn't free to apply, and two things limit it in this first preview.

The first is **null**. A `Point` variable can still be `null`, so a flattened slot needs an extra bit, a "null flag", to say "nothing here".⁵

The second is **atomicity** *≈ the guarantee that a read or write of a variable happens all at once, so another thread can never see half of an old value mixed with half of a new one*. Today, writing a reference to a field is atomic. If the JVM flattened a two-field value into an array slot and wrote it as two separate stores, another thread could read the new `x` with the old `y`: a point that never existed. That's called **tearing** *≈ seeing a mix of two different values because a multi-part write was observed half-done*, and the JVM won't allow it for value objects. So flattening is limited to what the hardware can read and write in one go: "on typical platforms, the available encoding may be as small as 64 bits including a null flag".⁵ The Valhalla project page puts it bluntly: "value objects with 64 bits or more of field data cannot normally be flattened."¹⁰ Bigger values still work correctly; they just get allocated as ordinary heap objects.⁵

Fixing both limits is deliberately left for later JEPs: null-restricted value types (letting you say "this field is never null", which frees the flag bit and relaxes atomicity) and 128-bit encodings are both out of JDK 28.⁷ So are **specialized generics** *≈ letting a generic type such as a list of Points store them flat instead of as references, which requires the JVM to keep type arguments at run time*. In JDK 28, an `ArrayList<Point>` is still an array of references.⁷

The JVM half: strict fields

JEP 539 is the plumbing underneath. In normal Java, a constructor can leak `this` before it finishes (by calling an overridable method, say), and someone can observe a `final` field still holding its default `0` or `null`. That's tolerable for identity objects. It's fatal for value objects, because `==` and flattening assume the fields are fixed from the moment the object exists.

So the JVM spec gets a new field flag, `ACC_STRICT_INIT`: a strictly-initialized field "must be assigned before it can be read", and must be assigned *before* the superclass constructor runs. If it's also `final`, it can't be assigned again afterwards.¹¹ The **bytecode verifier** *≈ the part of the JVM that checks a class file is well-formed and type-safe before running it* enforces this. InfoQ's summary: JEP 539 "supplies the bytecode verification that enforces the construction rules."⁵ One visible consequence for you: in a value class constructor, assign your fields before anything else touches `this`.

WHAT IT MEANS FOR YOU

In production, nothing changes yet. JDK 28 is a non-LTS release, the feature is preview-only and off by default, and Goetz expects it to still be in preview in JDK 29.² Nobody should rewrite a domain model around `value` this year. But two things are worth doing now, and one is worth watching.

Find out if you lock on wrappers today. The future breaking change, `synchronized` on an `Integer` throwing, is something you can detect on the JDK you already run. JEP 390's warnings have been in the JDK since 16. We compiled and ran this on OpenJDK 21.0.12.1:

```
Integer lock = 42;
synchronized (lock) { ... }

// $ javac -Xlint:all Demo.java
// warning: [synchronization] attempt to synchronize
//   on an instance of a value-based class

// $ java -XX:+UnlockDiagnosticVMOptions \
//       -XX:DiagnoseSyncOnValueBasedClasses=2 Demo
// [info][valuebasedclasses] Synchronizing on object
//   0x00000000c1b6abf0 of class java.lang.Integer
// [info][valuebasedclasses]   at Demo.main(Demo.java:13)
```

Setting the flag to `1` instead of `2` turns the log line into a fatal JVM error, which makes it a blunt but effective CI check. Locking on a boxed ID (`synchronized (userId)`) or on a `Boolean` field is the classic pattern to grep for. While you're there, search for `==` between wrapper types; that code is already wrong, and preview mode will change its behaviour.

Try it on real data structures in 2027. The likely early wins are the types you'd already describe as "just data": money amounts, coordinates, ranges, IDs, small tuples. If you have records like these, `value record` is a one-word experiment once JDK 28 builds are in hand. Measure with a profiler and a benchmark harness rather than trusting the 3x figure.

In the real world: the tools feel it first

Bytecode tools are the first to be hit by a change to the class-file format. JaCoCo, the widely used code-coverage library, has an open issue for JEP 401 support. Its testing found that simple value classes and records already work, but value classes with non-trivial constructor logic make ASM (the bytecode library JaCoCo is built on) throw an `IllegalArgumentException` while reading the new **stack map frames** *≈ metadata in a class file that tells the verifier what types are on the stack at each branch point*, and a reviewer warned the fix is "a huge patch".¹² Expect the same story with other agents, mocking libraries and bytecode-weaving frameworks: if you try value classes in 2027, update your tooling first.

Watch out for

- **Preview means the syntax may change.** The name has already changed once during this JEP's life. Code compiled with `--enable-preview` only runs on that exact JDK version with the flag on.
- **`==` is not your new `equals`.** The JEP still says use `equals`. And `String` is not a value class, so `==` on strings is still wrong.
- **Value objects can still be null.** Non-nullable value types are a separate, later JEP.
- **Not everything flattens.** Values with 64 bits or more of field data generally won't be flattened in this release; they still work, just without the memory win.
- **Security trade-off.** The design notes warn that `==` and `identityHashCode` "may indirectly expose private field values", and comparing two large trees of value objects can take unbounded time.⁵

WHAT HAPPENS NEXT

JDK 28 ships in March 2027.² Goetz has framed this as "just the first part of Valhalla",² and the project page lists what follows: null-restricted types, null-restricted value class types for denser layouts, enhanced primitive boxing, and a "parametric JVM" for specialized generics.⁹ If Goetz's expectation holds, value objects will still be a preview through the JDK 29 LTS, with the follow-on JEPs layering on top. The object model has changed, and the rest of Valhalla now has something to build on.

Words from this story

Project Valhalla: the long-running OpenJDK project to give Java objects the memory efficiency of primitives.

Identity: what makes an object distinguishable from every other object, even one with the same field values; what `==` checks today.

Object header: the bookkeeping bytes at the start of every object (lock state, GC info, class pointer).

Heap: the memory region where objects created with `new` live, managed by the garbage collector.

Reference: a pointer to an object on the heap; what a variable of a class type actually holds.

Cache line: the small chunk of memory (typically 64 bytes) a CPU loads at once.

Escape analysis: a JIT optimisation that avoids heap allocation for objects that never leave a method.

Value-based classes: JDK classes like `Integer`, `LocalDate` and `Optional` documented as "don't rely on identity".

Value class: a class declared with `value`; its instances have no identity and are compared by field values.

Contextual keyword: a word that is a keyword only in one position, so it can still be used as a variable name.

Reference type: a type whose variables hold references (or `null`).

Substitutability: two values are substitutable when no program can tell them apart.

Scalarization: the JIT splitting a value object into its fields in registers, so it's never allocated.

Heap flattening: storing a value's fields directly in the containing field or array slot.

Atomicity: a read or write happens all at once, never half-done.

Tearing: observing a mix of two values because a multi-part write was seen half-done.

Specialized generics: generic types that store their type arguments flat instead of as references.

Bytecode verifier: the JVM component that checks class files are well-formed before running them.

Stack map frames: class-file metadata telling the verifier the types on the stack at branch points.

Somewhere in your company there may be a Dockerfile that downloads a JDK from oracle.com, a golden VM image someone built in 2024, or a CI runner with "Java(TM)" in its version string. Until now, if that JDK was version 21, none of it mattered: Oracle's JDK 21 was free to run in production. On Tuesday 20 October that stops being true for every new update. Nothing breaks that day, no server goes down, and no build fails. That's exactly why this one is easy to miss.

WHAT HAPPENED

On 14 August, Donald Smith, Oracle's VP of product management for Java, published the formal notice: "all updates through and including September 2026 are available under the NFTC. Further Oracle JDK 21 updates, beginning with the October 2026 Critical Patch Update (CPU), are planned to be offered under the Java SE OTN license."¹³

Oracle's security calendar puts that **Critical Patch Update** *≈ Oracle's scheduled quarterly security release, published on a Tuesday in January, April, July and October* on **20 October 2026**, with the next ones on 19 January, 20 April and 20 July 2027.¹⁴ The release notes for the current Oracle JDK 21 build, 21.0.12.1 (18 August 2026), repeat the warning: updates "after September of 2026 will switch to the Java SE OTN license", and Oracle recommends against staying on 21.0.12.1 once the 20 October update is out.¹⁵

The same notice says where the free licence lives now: "JDK 25 updates are planned to remain available under the NFTC until October 2028."¹³

THE BACKGROUND YOU NEED

Two Oracle licences, two very different rules

Oracle publishes its JDK under two licences that matter here.

The **NFTC** *≈ Oracle No-Fee Terms and Conditions: the licence Oracle introduced in 2021 that lets anyone use Oracle JDK free of charge, including in production* is the generous one. Oracle's licensing FAQ says it "permits free use for all users – even commercial and production use", and redistribution "is permitted as long as it is not for a fee."¹⁶

The **OTN license** *≈ the Oracle Technology Network License Agreement for Java SE: free only for personal use, development, testing, prototyping and demos, not for running business applications in production* is the restrictive one. It is "free for Personal, Development and other Users only".¹⁷ Smith's post lists who can keep using the latest JDK 21 updates under it: "Java SE Universal Subscription customers, legacy Java SE product customers, users running on Oracle Cloud Infrastructure, those using Oracle Java as part of licensed Oracle products, and users running Java for personal use, development, testing, prototyping, or demonstrating applications."¹³ Notice what's missing from that list: your company's production servers running your own application.

How we got here: three moves since 2019

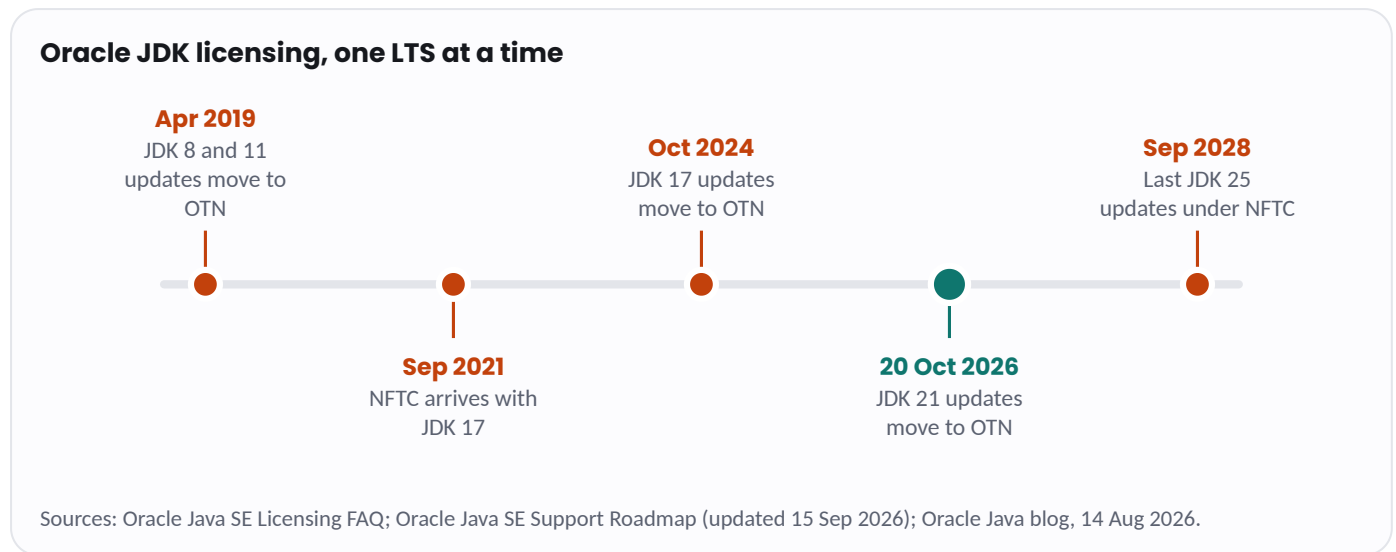
If this feels familiar, it's because it has happened before, twice.

April 2019. Oracle moved JDK 8 and 11 updates to the OTN licence.¹⁶ Free production use of Oracle's builds of those versions ended.

September 2021. Oracle introduced the NFTC with JDK 17,¹⁶ bringing free production use back, with a built-in time limit. An LTS release's updates stay under NFTC "until... one year after the release" of the next LTS. For JDK 21, that meant "until September of 2026, one year after the release JDK 25 LTS, in September 2025."¹⁶

October 2024. JDK 17 hit that limit. Its updates moved to OTN, joining 8 and 11.¹⁷

October 2026. Now JDK 21 follows the same path. It's the rule from 2021 working as designed.



Oracle JDK is not "Java"

This is the most important idea in the story. The Java platform is open source. **OpenJDK** *≈ the open-source project, and codebase, where Java itself is developed, licensed under GPLv2 with the Classpath Exception* is the source code. Oracle JDK is *one company's build* of that code, with Oracle's licence attached. Many other organisations build the same source and give it away under the open-source licence, which is **GPLv2 with Classpath Exception** *≈ the GNU General Public License v2 plus an exception that lets you run your own (even proprietary) code on the JDK without your code becoming GPL*.

So the licence change applies to the Oracle *binary*, not to Java 21 itself. A JDK 21 from another vendor isn't affected at all.

HOW IT WORKS

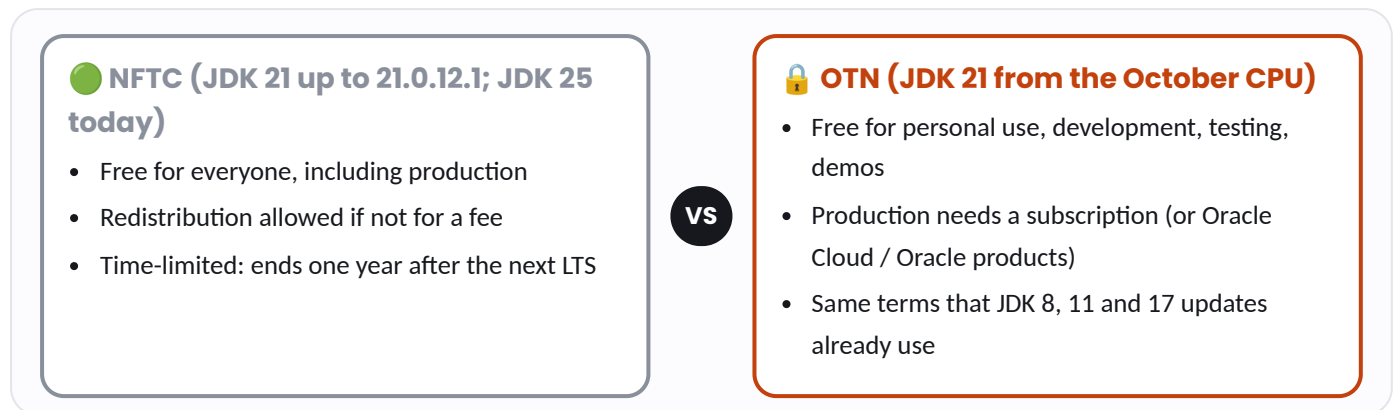
What actually changes on 20 October

Two things that sound similar behave very differently.

The builds you already have keep their licence. Oracle's FAQ says: "You may continue to use releases you have downloaded under the terms of the license under which you downloaded them."¹⁶ If you downloaded Oracle JDK 21.0.12.1 under NFTC, you can keep running it in production indefinitely.

The next update comes under OTN. The October CPU build of Oracle JDK 21, and every Oracle JDK 21 update after it, is OTN-licensed.¹³ Running it in production requires you to be in one of the permitted categories, which for most companies means a paid subscription.

That leaves a trap. Staying on 21.0.12.1 is legal, but from 20 October it no longer gets security fixes, and Oracle itself advises against using it after that date.¹⁵ So for a production system on Oracle JDK 21, "do nothing" quietly becomes "run an unpatched JDK".



Your four options

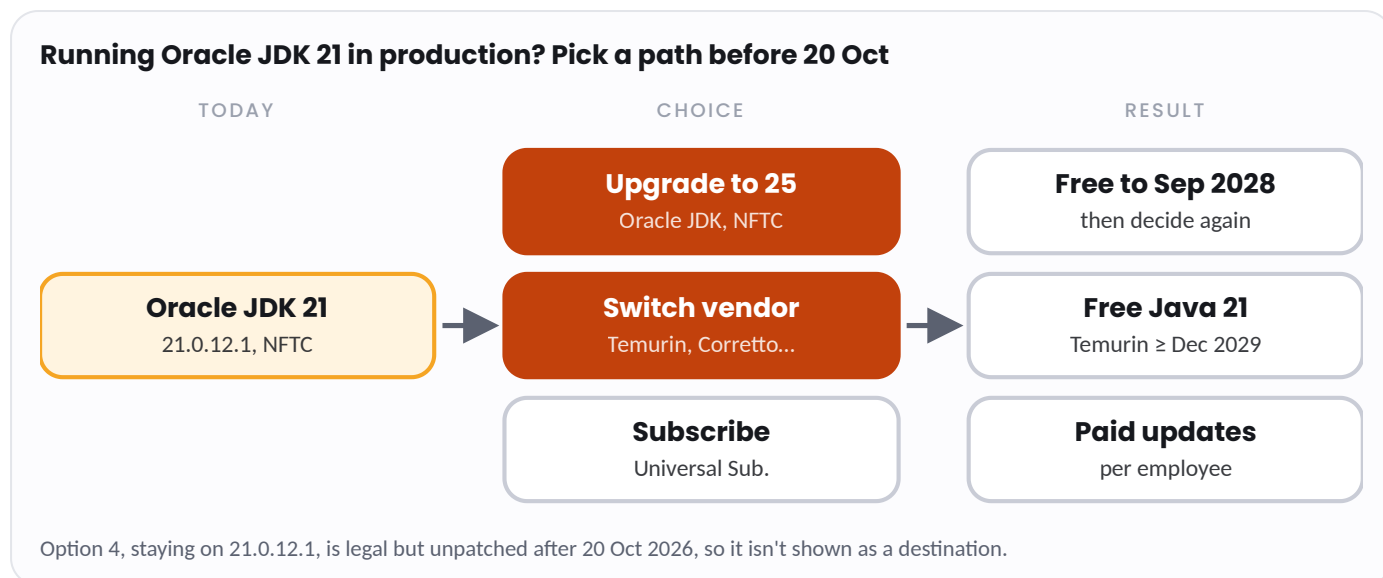
There are really only four ways to respond, and each has a clear cost.

1. Upgrade to Oracle JDK 25. It's the current LTS, and its updates are free under NFTC until September 2028, "one year after the next release that Oracle currently plans to offer long term support for, which is Java 29 in September 2027."¹⁶ Oracle's roadmap says it plainly: "Users wanting to continue with permissively licensed versions of Java after October of 2026 should upgrade to Oracle JDK 25 or later."¹⁷ The cost is an LTS-to-LTS upgrade: four feature releases of changes to test. And it buys you two years, not forever. You'll face the same decision for JDK 25 in 2028.

2. Switch to another vendor's free JDK 21. This changes the least code: it's the same Java 21, built from the same OpenJDK source, from someone else. Eclipse Temurin (from the Adoptium project) is free and promises JDK 21 builds until "at least Dec 2029".¹⁸ Amazon Corretto is "distributed by Amazon under an Open Source license at no cost to you", under GPLv2 with the Classpath Exception, with Corretto 21 supported until October 2030.¹⁹ Your Linux distribution's OpenJDK package is another route; the sandbox we used for this issue runs Ubuntu's OpenJDK 21.0.12.1. Note one thing Oracle's own free builds won't do for you here: the GPL-licensed Oracle OpenJDK builds at jdk.java.net only get updates for about six months per release,¹⁶ so they're a way to run the newest JDK, not a way to stay on 21.

3. Pay Oracle. The Java SE Universal Subscription covers JDK 21 updates, with Oracle's roadmap showing Premier Support to September 2028 and Extended Support to September 2031.¹⁷ It's priced per employee, not per server or per developer. According to Azul's FAQ (a competitor, so read it as such), the metric "starts at \$15 per month per employee" and counts full-time, part-time and temporary staff and contractors.²⁰ For a large company running a handful of Java services, that difference matters a lot.

4. Freeze on 21.0.12.1. It's free and legal, but it gets no more security fixes. This is only a stopgap while you carry out option 1 or 2.



WHAT IT MEANS FOR YOU

Step one is finding out whether you're affected, and most teams aren't. If your containers start FROM `eclipse-temurin`, `amazoncorretto` or a distro image with its own OpenJDK, this change doesn't touch you. The risk is concentrated in places nobody has looked at for a while: VM images, on-prem servers, desktop tooling deployed to the business, vendor appliances, CI agents.

Checking is a one-liner on each machine. The vendor shows up in the system properties:

```
$ java -XshowSettings:properties -version 2>&1 | grep -E "java.vendor =|runtime.name|java.version ="
java.runtime.name = OpenJDK Runtime Environment
java.vendor = Ubuntu
java.version = 21.0.12.1
```

That's the output from our sandbox. An Oracle JDK reports Oracle as the vendor. Run this across your fleet (or ask your inventory/SBOM tooling for "java.vendor"), then sort what you find into production and non-production. Remember that development and testing are explicitly allowed under OTN,¹³ so a developer laptop with the October Oracle JDK 21 update is fine. The same binary on a production server is the problem.

Step two is choosing on engineering grounds, not under deadline pressure. For most teams the cleanest answer is option 2 now (a drop-in vendor swap, same version, re-run your tests) and option 1 on your normal schedule. Swapping a JDK 21 build for another JDK 21 build is about as low-risk as infrastructure changes get, since both come from the same OpenJDK source. The things to check are mostly operational: where the binaries come from, how your images get patched each quarter, and whether anything depends on Oracle-specific packaging or installers.

And put 2028 in the calendar now. If you move to Oracle JDK 25 to stay on NFTC, the same rule ends its free updates in September 2028.¹⁶ Either make vendor choice a deliberate decision now, or plan to make this move every two years.

Watch out for

- **"Java 21 is no longer free" is wrong.** Only Oracle's JDK 21 updates change licence. Java 21 from Temurin, Corretto, your Linux distro and other OpenJDK builds is unaffected.
- **Your old download is fine; your next patch isn't.** Builds you downloaded under NFTC stay under NFTC. The October update and everything after is OTN.
- **Unpatched isn't a free option.** Freezing on 21.0.12.1 means no security fixes after 20 October.
- **Dev and test are still free under OTN.** The restriction is about production use, so don't panic-uninstall developer JDKs.
- **Some environments are covered.** Oracle Cloud Infrastructure and Oracle products that bundle Java are on Oracle's list of users who can keep using the updates.¹³ Check your specific contract.
- **This is not legal advice.** Licence terms are contracts. If you're unsure whether a deployment counts as "production", ask whoever owns software licensing at your company.

WHAT HAPPENS NEXT

The October CPU lands on **Tuesday 20 October**; the January one follows on 19 January 2027.¹⁴ Two days after Oracle's CPU, on 22 October, Spring runs its first monthly patch train under its new release model,²¹ so for many teams the week of 19 October was already going to be an upgrade week. If you have Oracle JDK 21 in production, put the JDK decision in the same sprint. The bigger lesson from 2019, 2024 and now 2026 is that "which JDK do we run?" deserves an owner and a written answer, so the next licence change becomes a calendar entry, not a surprise.

Words from this story

Critical Patch Update (CPU): Oracle's scheduled quarterly security release, in January, April, July and October.

NFTC: Oracle No-Fee Terms and Conditions; free use of Oracle JDK including production, for a limited time per release.

OTN license: Oracle Technology Network License for Java SE; free for personal use, development, testing and demos, not general production.

OpenJDK: the open-source project and codebase where Java is developed; every JDK distribution is built from it.

GPLv2 with Classpath Exception: the open-source licence of OpenJDK, which lets you run proprietary code on it without that code becoming GPL.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Spring's monthly "Patch Thursday"**: the whole Spring portfolio now ships on one day each month (the Thursday after the third Monday), with the first patch train on 22 October, a response to nearly 80 security reports a month.²¹
- **Jackson CVE-2026-68497**: a few megabytes of crafted JSON aimed at a `Duration` or `XMLGregorianCalendar` field can burn minutes of CPU (CVSS 7.5); fixed in 2.18.10, 2.21.6, 2.22.2, 3.1.6 and 3.2.2.²²
- **Quarkus 4.0.0.Beta1 (1 Oct)**: Java 21 baseline, Vert.x 5, HTTP/3 over QUIC, Hibernate ORM 8, Jackson 3 and CDI 5.0, with GA planned for the end of November.²³
- **JEP 543, Structured Concurrency (final)**: the API for treating a group of related tasks as one unit of work became a Candidate to leave preview in JDK 28.²⁴
- **JEP 544, Ahead-of-Time Code Compilation**: Project Leyden's plan to ship optimised native code at JVM startup was Proposed to Target JDK 28, with its review ending 28 September.²⁵
- **Two ZGC JEPs, 545 and 546**: faster ZGC startup and adaptive heap sizing both became Candidates for a future JDK.²⁵
- **Build tools**: Gradle 9.8.0 added Java 27 support for the daemon and toolchains, and Apache Maven 4.0.0-RC7 continued the march toward Maven 4 GA.²⁵

1. **JEP targeted to JDK 28: 401: Value Objects (Preview)**, Inside.java, 20 Sep 2026.
2. **Java's Project Valhalla finally lands a preview in JDK 28**, The Register, 15 Jun 2026.
3. **8389219: Implement JEP 401: Value Objects (Preview), PR #31120**, openjdk/jdk on GitHub.
4. **Java News Roundup: OpenJDK JEPs, Jakarta EE, GraalVM, TornadoVM, Micronaut, Quarkus, JobRunr, Maven**, InfoQ, 3 Aug 2026.
5. **Project Valhalla's First Preview: JEP 401 Redefines == for Java Objects**, A N M Bazlur Rahman, InfoQ, 10 Aug 2026 (summarising **JEP 401**).
6. **JEP draft 8277163: Value Objects (Preview)**, OpenJDK, Dan Smith, 2021–2023. An earlier, now-closed design draft of JEP 401, quoted for motivation and history.
7. **Project Valhalla, Explained: How a Decade of Work Arrives in JDK 28**, JVM Weekly vol. 180, 2026.
8. **Try JEP 401 Value Classes**, Inside.java, 27 Oct 2025; build from jdk.java.net/valhalla.
9. **Project Valhalla**, OpenJDK project page.
10. **Value Classes and Objects**, OpenJDK Project Valhalla.
11. **Strict Field Initialization in the JVM (JVMS changes)**, JDK 28 early-access specs.
12. **Add support for Java 28 preview feature: Value Classes and Objects (JEP 401), issue #2164**, JaCoCo on GitHub.
13. **JDK 21 approaches end of permissive license**, Donald Smith, Oracle Java blog, 14 Aug 2026.
14. **Critical Patch Updates, Security Alerts and Bulletins**, Oracle.
15. **JDK 21.0.12.1 release notes**, Oracle, 18 Aug 2026; see also **21.0.12 release notes**, 21 Jul 2026.
16. **Oracle Java SE Licensing FAQ**, Oracle.
17. **Oracle Java SE Support Roadmap**, Oracle, updated 15 Sep 2026.
18. **Temurin Support**, Eclipse Adoptium.
19. **Amazon Corretto FAQs**, Amazon Web Services.
20. **Java 21 End of Free Commercial Use FAQ**, Azul (a JDK vendor competing with Oracle).
21. **Releasing Spring for Modern Challenges**, spring.io, 21 Sep 2026.
22. **CVE-2026-68497**, Tenable, published 11 Sep 2026.

23. **Quarkus 4.0.0.Beta1 released**, quarkus.io, 1 Oct 2026.
24. **Java News Roundup: New OpenJDK JEPs, CDI 5.0, Spring, Open Liberty, RefactorFirst, ADK for Kotlin**, InfoQ, 14 Sep 2026.
25. **Java News Roundup: TornadoVM 7.0, Groovy 6.0, GraalVM, Hibernate, Quarkus, Gradle, Maven**, InfoQ, 28 Sep 2026.